ELSEVIER

Parallel Computing 22 (1997) 1563–1578

PARALLEL COMPUTING

Parallel matrix product algorithm in the de Bruijn network using emulation of meshes of trees [1]

Dominique Barth [2]

LRI U.R.A. CNRS 410, Université de Paris Sud, Centre d'Orsay, Bât. 490, 91100 Orsay, France

Received 15 March 1996; revised 25 May 1996, 9 October 1996

Abstract

We present an algorithm computing the product of two square matrices in the de Bruijn network by using emulation: the algorithm is based on a $\lfloor (d-1)/2 \rfloor$-embedding of meshes of d-ary trees into de Bruijn graphs and also on a mapping of particular trees.

Keywords: Matrix computation; Emulation; Embedding; Mesh of d-ary trees; De Bruijn graph

1. Introduction

In this paper, we deal with graphs modeling interconnection networks of parallel machines architectures (see [3] for graphs notations and definitions). We focus on the capacity of implementing good parallel algorithms in such networks. In particular, we study the emulation of a network by another, consisting in simulating in a network the execution of parallel algorithms in other networks.

Many different works about emulation involve graph embeddings (definition is given in the following) or routing of permutations [14,18,15,19]. This gives informations about how the structure of a network can be emulated by another one. With the particular case we deal with in this paper, we want to show that the emulation of different networks by a network G can be an adjusment tool to obtain an efficient parallel algorithm in G. This emulation takes care of structural embeddings and mappings in G; it also depends on different properties of the particular algorithms which must be emulated (initial and final location of data, vertices really used,...).

[1] This work was supported by the "Opération RUMEUR" of the French PRC PRS.

[2] Email: barth@lri.fr.

0167-8191/97/$17.00 © 1997 Elsevier Science B.V. All rights reserved
PII S0167-8191(96)00064-6

We focus here on the de Bruijn graph. This graph has been proposed as an interesting topology of parallel machine architecture in terms of communication capabilities, fault tolerance,... [5]. But there are not many parallel algorithms developed in it [8,20]. Thus, we study a classical example: we implement a parallel algorithm realizing the product of two square matrices on the de Bruijn graph. On one hand, it is difficult to determine the initial location of data in the network to obtain a good algorithm. On the other hand, in many other networks, one uses the emulation of the grid [10,18,19], in which such a good algorithm has been developed [11]; but the presently known dilation and congestion of the embedding of the grid into the de Bruijn graph are too bad to use a similar emulation on it [13].

Our purpose is double. We first show with this example that a good parallel algorithm can be developped in the de Bruijn graph. Secondly, this example shows how emulation can be used to obtain efficient algorithms.

We deal here with emulation of meshes of d-ary trees [2] by the de Bruijn graph, based on some graph embeddings. An embedding of a graph G into a graph H, with $|V(G)| \leqslant |V(H)|$, is a couple (p, R) where p is an injective mapping from $V(G)$ into $V(H)$, and R is an application which associates to each edge $[x, y]$ of G a chain between $p(x)$ and $p(y)$ denoted by $R(x, y)$ in H. Some parameters, defined as follows, are used to evaluate an embedding:

- *The expansion*, equal to $|V(G)|/|V(H)|$.
- *The dilation*, equal to $\max_{[x,y]\in E(G)}(|R(x, y)|)$ where $|R(x, y)|$ is the length (i.e. the number of edges) of the path.
- *The congestion of an edge* $[u, v]$ of G is $c(u, v) = |\{[x, y] \in E(G) | [u, v] \in R(x, y)\}|$. *The congestion* of the embedding is equal to $\max_{[u,v]\in E(H)} c(u, v)$.

A *k-embedding* of a graph G into a graph H, with $k \cdot |V(G)| \leqslant |V(H)|$, consists in embedding into H a graph made of k disjoint copies of G [1]. Finally, a *mapping* of G on H is a couple of applications (p, R) defined as before, but where p is not necessary an injective application.

Embedding and k-embedding are used to give the following result on emulation. Under a link-bound model of communication [12], a communication step in a graph consists for some vertices to exchange data with some neighbours (see Section 2.1).

Proposition 1. *If G can be embedded (or k-embedded) into H with a dilation d and a congestion c, then any communication step in G can be emulated into H*

- *in $d \cdot c$ steps or less* [19],
- *in $O(d + c)$ steps* [16].

Under a linear time model [12], this proposition can apply if the communications are scheduled in different consecutive steps, even if the real time of each step is different. Fortunately, our algorithm in the mesh of d-ary trees respects this hypotheses.

In Section 2, we give graph definitions. In Section 3, we specify the communication model we use and we present the performances of our final algorithm. In Section 4, we give a general matrix product algorithm in the mesh of d-ary trees. We use it to define a

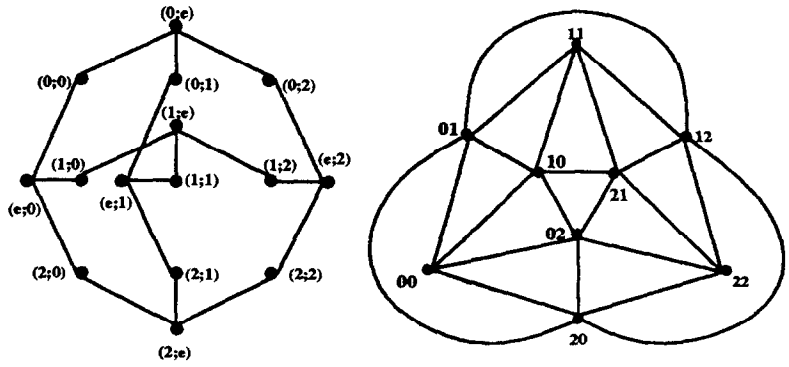


Fig. 1. $MT(3, 1)$ and $UB(3, 2)$.

new algorithm in the de Bruijn graph in Sections 5 and 6, by giving some results about embeddings in this graph.

2. The mesh of d-ary trees and the de Bruijn graph

In the following, we use some language theory notations from [17]. Let $\{0, \ldots, d-1\}$ be an alphabet and v be a word on it. We denote by $|v|$ the *length* of v, i.e. the number of letters in v; we denote by e the empty word, i.e. $|e| = 0$. Each letter $x \in \{0, \ldots, d-1\}$ is also considered as an element of $\mathbb{Z}_d$. For each word $v = x_1 \ldots x_k$ of length k in $\{0, \ldots, d-1\}^k$, we denote by $[u]_d$ the integer $x_1.d^{k-1} + x_2.d^{k-2} + \cdots + x_k$.

2.1. The mesh of d-ary trees $MT(d, n)$

We first define the *complete d-ary tree of depth n* denoted by $T(d, n)$.

- The set of vertices is the set of all the words of length at most n of $\{0, \ldots, d-1\}^*$.
- A pair of vertices $[m, m']$ is an edge of $T(d, n)$ if $m = m'x$ (or $m' = mx$), with $x \in \{0, \ldots, d-1\}$. Then m' is the *parent* of m and m is a *child* of m'. The *internal vertices* of $T(d, n)$ are all the vertices except the leaves.

The mesh of d-ary trees, introduced in [2], is a generalization of the mesh of binary trees (i.e. with $d = 2$) [15]. We first give a construction of $MT(d, n)$ using two well-known graphs: the square grid $G(d^n, d^n)$ having d^n lines and d^n columns, and the complete d-ary tree of depth n. As usually, a vertex (i, j) of $G(d^n, d^n)$ is in line i and column j in the grid. On each line i (resp. column j) of the grid, we identify (i, j) with the leave m of a complete d-ary tree of depth n such that $[m]_d = j$ (resp. $[m]_d = i$). Then, we delete the edges linking vertices in the initial grid.

The tree isomorphic to $T(d, n)$ which leaves are identified with vertices of the line i (resp. column j) of the grid is called the *line-tree i* (resp. *column-tree j*).

We now give a definition of $MT(d, n)$ which uses an alphabet (Fig. 1).

- The vertices of $MT(d, n)$ are all the couples denoted by $(u; v)$ of words of $\{0, \ldots, d-1\}^*$, such that $|u| = n$ and $|v| \leqslant n$, or $|v| = n$ and $|u| \leqslant n$.

- There is an edge linking two vertices $(u; v)$ and $(w; z)$ in $MT(d, n)$ if either $|u| = n$, $u = w$ and $[v, z] \in E(T(d, n))$, or $|v| = n$, $v = z$ and $[u, w] \in E(T(d, n))$. This edge is denoted by the pair $[(u; v), (w; z)]$.

The number of vertices of $MT(d, n)$ is $|V(MT(d, n))| = d^n\ (d^n + 2(d^n - 1)/(d - 1))$ and the number of edges is equal to $2d^n((d^{n+1} - 1)/(d - 1) - 1)$. The maximal degree is equal to d if $n = 1$, equal to $d + 1$ otherwise, and the diameter is equal to $4n$ [2].

For each word u of $\{0, \ldots, d - 1\}^n$, the vertices $\{(u; v) | |v| \leqslant n\}$ (resp. $\{(v; u) | |v| \leqslant n\}$) in $MT(d, n)$ are the ones of the line-tree (resp. column-tree) $[u]_d$, which we also denote $T_{(u; e)}$ (resp. $T_{(e; u)}$) since $(u; e)$ (resp, $(e; u)$) is the root of this tree.

2.2. The de Bruijn graph UB(d, n)

The undirected de Bruijn graph $UB(d, n)$ is defined as follows (Fig. 1).

- The vertices of $UB(d, n)$ are all the words of $\{0, \ldots, d - 1\}^n$.
- A pair of different vertices $[x_1 \ldots x_n, y_1 \ldots y_n]$ is an edge of $UB(d, n)$ if $x_1 \ldots x_{n-1} = y_2 \ldots y_n$ or if $x_2 \ldots x_n = y_1 \ldots y_{n-1}$.

This graph has d^n vertices, a maximal degree equal to $2d$ and a diameter equal to n.

The de Bruijn graph models here an *homogeneous* network: each vertex of $UB(d, n)$ represents a node containing both a router and a processor with local memory in the corresponding architecture.

3. Model of execution of a parallel algorithm

3.1. Communication and computation hypothesis

The model of execution on a network of a parallel algorithm we consider here is similar to the one of [6]. The communication hypotheses and notation are the ones of [12].

(1) The communications in the network respect the hypotheses *link bound* and *linear time*:
   - Each vertex can communicate with all of its neighbors at each time.
   - The time required for two incident vertices to exchange a message of length L is equal to $\beta + L\tau$, where β is the start up time of the communication, and τ is the propagation time on the link. A message of length 1 contains one integer.

(2) The time required to perform an elementary operation (addition or multiplication of two integers) is t_{op}.

(3) Arithmetic and communication cannot overlap.

(4) Initially, there is no duplication of data in the network.

The execution time of an algorithm in a network under these hypotheses is a function of β, τ and t_{op}. Moreover, to apply Proposition 1, communications realised in the target network must be scheduled in communication steps, possibly of different duration, as we do in Section 4.

The aim of the above model is not to accurately represent the exact behaviour of a parallel machine. It is rather to allow us to evaluate the emulation properties of a network, both by using Proposition 1 and by giving a clear description of the algorithms to be emulated.

3.2. Performances of the final algorithm

Under the above hypothesis, we give in Section 5 an algorithm computing the product of two matrices of order $N = 2kd^n$, with $k^2 = \lfloor (d-1)/2 \rfloor$, on the de Bruijn graph $UB(d, n)$. The time needed to execute this algorithm is in $\theta(dNt_{op} + (k \cdot \log_d N)\beta + N\tau)$. This algorithm is good in comparison with efficient ones developped on the hypercube or on other networks [6,15,19]; for example, on a hypercube of order N^2, the matrix product obtained in [6] needs a time $2N\tau + 2Nt_{op} + (N^2 - N)\beta$. Note that in the actual architectures β is a much greater parameter than τ and t_{op}.

Our algorithm is also better than the ones we can obtain by emulating on the de Bruijn graph classical matrix product algorithms running on the torus, as the Fox's or the Cannon's algorithms [11,9], by considering the better known dilation and congestion of an embedding of the Torus in the de Bruijn graph [13]. It also seems difficult to directly implement this algorithms on the de Bruijn graph. They need non trivial global communication schemes, like broadcastings of the diagonals of the matrices and rollings of lines and columns; the time needed to realise this communication depends on the initial location of data of the matrices in $UB(d, n)$.

Finally, the algorithms we give might also lead to the implementation of new algorithms in other communication models (like the *circuit switching* model for example [19]).

4. A parallel algorithm in $MT(d, n)$

4.1. A first algorithm in MT(d, n)

We first present a parallel algorithm computing the product of two square matrices of dimension d^n in $MT(d, n)$, under a link bound model of communication. In all the following, we consider n much greater than d. Let A and B be two square matrices of order $N = d^n$. We denote by $A[i, j]$ the element in line i and column j of A and by $A[i, .]$ (resp. $A[., j]$) the vectors corresponding to line i (resp. column j) of A.

Initially, each vertex $(u; v)$ of $MT(d, n)$, with $|u| = |v| = n$, $[u]_d = i$ and $[v]_d = j$, contains elements $A[i, j]$ and $B[j, i]$; these data are considered in the following as vectors of dimension 1. At the end of the execution, $(u; v)$ contains $M[i, j]$.

This first algorithm computes $M = A \times B$ in $MT(d, n)$; it never uses simultaneously line trees and column trees, and the computation steps mainly consist in summing vectors. Consider u and v two words of length n, with $[u]_d = i$ and $[v]_d = j$. The algorithm is presented in three parts.

- First, the elements of $B[j, .]$ are broadcasted to all the leaves of column tree j (Figs. 2(a) and (b)) by using this tree.

- Then, in each leaf $(u; v)$, vector $M_j[i, .] = A[i, j] \times B[j, .]$ is computed and all the vectors $M_j[i, .]$ are summed by going back up in line tree i (Fig. 2(c)). The resulting vector is $M[i, .]$.
- At the end, $(u; e)$ (with $[u]_d = i$) contains $M[i, .]$ which is distributed to the leaves of line tree i (Fig. 2(d)).

We present now these three parts more in details in order to compute the time needed to execute this algorithm. Note that, to lighten the writing, we frequently give only upper bounds, function of $N = d^n$, of the time needed by each part.

Part 1: *Allocation of data.*

The purpose of this part is to make $(u; v)$, with $|u| = |v| = n$, contain $B[j, .]$, with $j = [v]_d$.

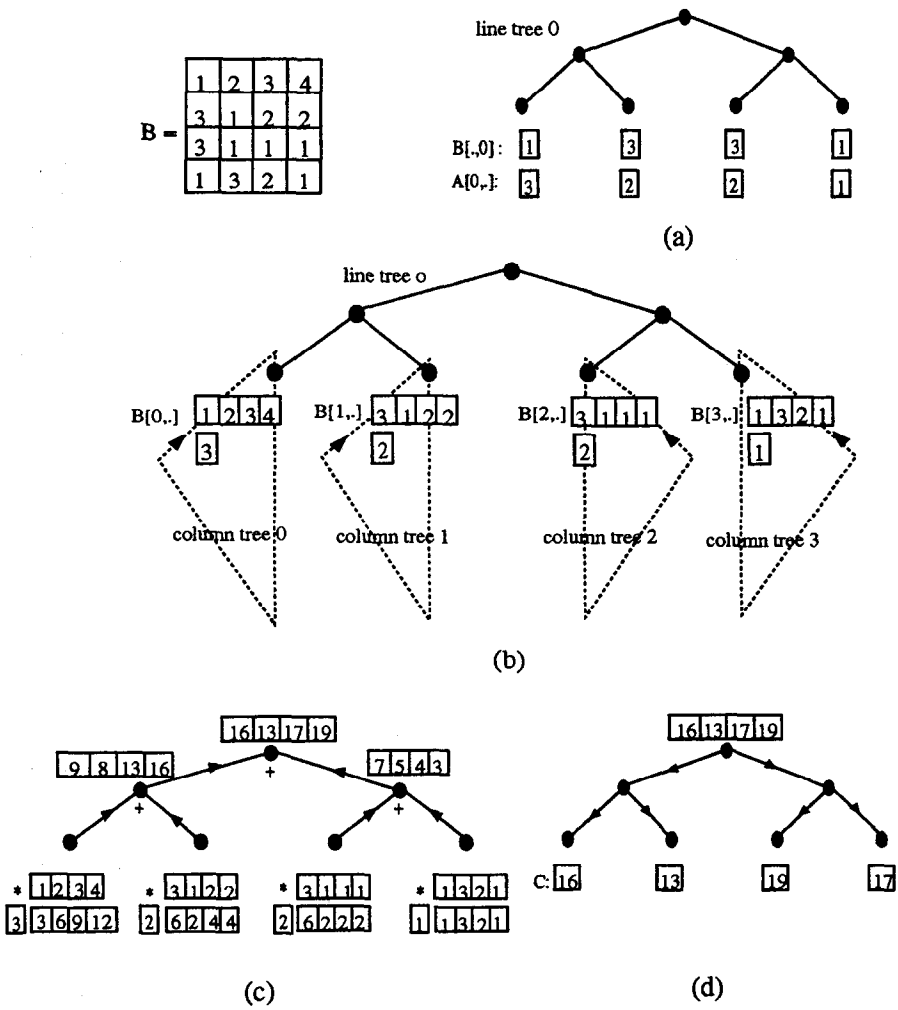


Fig. 2. Example of computation in a line tree of $MT(2, 2)$.

1.1. *In column tree j, each vertex* $(w; v)$, *with* $0 < |w| < n$, *receives a vector of dimension* $d^{n-|w|-1}$ *from each of its d sons. It then makes the concatenation of these vectors and send this new one of length* $d^{n-|w|}$ *to its parent if* $|w| > 0$. *At end, the root* $(e; v)$ *contains* $B[j, .]$ *at time*

$$t_1 = n\beta + \tau\left(\sum_{k=0}^{n-1} d^k\right) = n\beta + \frac{d^n - 1}{d-1}\tau.$$

1.2. *Now* $B[j, .]$ *is broadcasted from* $(e; v)$ *to all the leaves of* $T_{(e; v)}$. This needs a time t'_1. To evaluate t'_1, we consider the following result of [4], which respects the hypothesis of Section 3.1.

Proposition 2. *Let h be the depth of a spanning tree rooted at node r of a graph G. There exists a link-bound broadcasting algorithm from node r whose time is* $(\sqrt{L\tau} + \sqrt{(h-1)\beta})^2$, *where L is the message length.*

As a corollary, we can determine a communication scheme such that

$$t'_1 \leqslant \left(\sqrt{d^n\tau} + \sqrt{(n-1)\beta}\right)^2 \leqslant 2d^n\tau + 2(n-1)\beta.$$

Thus, Part 1 needs a time $T_1 = t_1 + t'_1$, i.e.

$$T_1 \leqslant (3n-2)\beta + \frac{d^n(2d-1)-1}{d-1}\tau \leqslant 3(\log_d N)\beta + 2N\tau.$$

Part 2: *Computing* $M = A \times B$.

In this part, we use a pipeline computation algorithm: M is computed by blocks from the leaves to the root of each line tree. We first define n subvectors of each line $B[j, .]$, denoted by B_j^α with $0 \leqslant \alpha \leqslant n-1$, of length d^n/n (we consider here that n is a divisor of d^n; else, we take n subvectors of length $\lceil d^n/n \rceil$, except for the last one). B_j^α is defined by

$$\forall a \in \{0, \ldots, d^n/n - 1\}, \quad B_j^\alpha[a] = B[j, \alpha n + a].$$

Part 2 takes $2n$ phases, each one composed of computing steps and communication steps.

Phase 0:

- *Each leaf* $(u; v)$ *of* $T_{(u; e)}$ *computes the vector* $N_{i,j}^0 = B_j^0 \times A[i, j]$.
- *Then,* $(u; v)$ *sends* $N_{i,j}^0$ *to its parent in* $T_{(u; e)}$.

Phase k, with $1 \leqslant k \leqslant 2n-2$:

- *If* $k \leqslant n-1$ *then each leaf* $(u; v)$ *computes* $N_{i,j}^k = B_j^k \times A[i, j]$.
- *In parallel, each internal vertex of* $T_{(u; e)}$ *which has received one vector from each of its children in phase* $k-1$ *makes the sum of all these vectors.*
- *Each vertex, different from* $(u; e)$, *which has computed a new vector sends it to its parent.*

Phase $2n-1$: *Each vertex* $(u; e)$ *adds the different vectors it has received from its children at step* $2n-2$. *It finally contains* $M[i, .]$.

The time required by this part is:

- For phase 0, $(d^n/n)t_{op} + (\beta + (d^n/n)\tau)$.
- For all the phases k, with $1 \leqslant k \leqslant 2n-2$,

$$(2n-2)\left(\frac{d^{n+1}}{n}t_{op} + \left(\beta + \frac{d^n}{n}\tau\right)\right).$$

- The phase $2n-1$ needs a time $(d^{n+1}/n)t_{op}$.

Thus, the global time for Part 2 is

$$T_2 = \left(\frac{(2n-1)d^{n+1} + d^n}{n}\right)t_{op} + (2n-1)\beta + \frac{2n-1}{n}d^n\tau$$

$$\leqslant \left(2dN + \frac{N}{\log_d N}\right)t_{op} + 2(\log_d N)\beta + 2N\tau.$$

Part 3: *Distribution.*

Each element of $M[i, .]$, *initially contained by* $(u; e)$ *with* $[u]_d = i$ *and* $|u| = n$, *is sent to its final destination: the element* $M[i, j]$ *is sent to* $(u; v)$, *with* $|v| = n$ *and* $[v]_d = j$.

This needs time

$$T_3 = n\beta + \frac{d^n - 1}{d-1}\tau \leqslant (\log_d N)\beta + \frac{N}{d-1}\tau.$$

Hence, the global time needed to compute this algorithm in $MT(d, n)$ is $T_g = T_1 + T_2 + T_3$, with

$$T_g \leqslant \left(2dN + \frac{N}{\log_d N}\right)t_{op} + 6(\log_d N)\beta + \left(\frac{4d-3}{d-1}\right)N\tau.$$

Since this algorithm never uses simultaneously internal vertices in line trees and internal vertices in column trees of $MT(d, n)$, we can compute $M = A \times B$ and another product of two such matrices $M' = A' \times B'$ in $MT(d, n)$, by running simultaneously the above algorithm in the line trees on one hand and in the column trees on another hand. Thus, we run the above algorithm on A and B and in parallel a similar algorithm on A' and B'. This new algorithm uses line trees in Part 1, and column trees in Parts 2 and 3. Hence, initially, each vertex $(u; v)$, with $|u| = |v| = n$, contains $A'[j, i]$ and $B'[i, j]$ where $i = [u]_d$ and $j = [v]_d$. This algorithm consists in

- First, the elements of $A'[., i]$ are broadcasted to all the leaves of line tree i.
- Then, in each leaf $(u; v)$, vector $M'_i[., j] = B'[i, j] \times A'[., i]$ is computed and all the vectors $M'_i[., j]$ are summed by going back up in column tree j. The resulting vector is $M'[., j]$.
- At the end, $(e; v)$ contains $M'[., j]$ which is distributed to the leaves of column tree j.

The only execution of this algorithm needs a time T_g.

To run simultaneously these two algorithms, it is easy to see that all the communication steps can be scheduled in line and column trees so that there is no loss of time

compared to T_g. Moreover during Part 2, for each phase $p \leqslant n-1$, each leaf $(u; v)$ has now to compute $N_{i,j}^p$ and $N_{j,i}'^p = B'[i, j] \times A_i'^p$, which takes a time $2(d^n/n)t_{op}$. If $p > 0$ then at each phase, some other vertices realize a computation step needing a time $(d^{n+1}/n)t_{op}$, which is greater than or equal to $(2d^n/n)t_{op}$. Hence there is only an additional time for $p = 0$, and so the two products can be simultaneously realized in a time $T_G = T_g + (d^n/n)t_{op}$. In the following, we call respectively $\mathcal{M}_{line}$ and $\mathcal{M}_{col}$ these two algorithms.

4.2. A global algorithm in MT(d, n)

We finally use simultaneously the algorithms $\mathcal{M}_{line}$ and $\mathcal{M}_{col}$ we obtained, to realize the product of two square matrices A and B of order $2d^n$; each matrix is considered composed of four disjoint square submatrices of order d^n (see Fig. 3), each one denoted by A_α and B_α, where $0 \leqslant \alpha \leqslant 3$. Initially, each vertex $(u; v)$ in $MT(d, n)$, where $|u| = |v| = n$, $[u]_d = i$ and $[v]_d = j$, contains elements.

- in line i and column j, elements from submatrices A_1, A_3, B_3 and B_4,
- in line j and column i, elements from the other submatrices.

The resulting matrix $M = A \times B$ is also considered composed of four submatrices M_α.

First $M_1 = (A_1 \times B_1) + (A_2 \times B_3)$ is computed; as we have seen, the two products are simultaneously realized in a time T_G using respectively $\mathcal{M}_{line}$ and $\mathcal{M}_{col}$, and the sum is done in a time t_{op} in vertices $(u; v)$ with $|u| = |v| = n$. The three other submatrices in M are computed by the same way:

$$M_1 = A_1 \times B_1 + A_2 \times B_3, \qquad M_3 = A_3 \times B_1 + A_4 \times B_3,$$
$$M_2 = A_1 \times B_2 + A_2 \times B_4, \qquad M_4 = A_3 \times B_2 + A_4 \times B_4.$$

The global time required is at most equal to $4(T_G + t_{op})$. It can be improved as follows. To compute M_1 and M_3 (resp. M_2 and M_4), we execute twice Part 1 of algorithm $\mathcal{M}_{line}$. These two executions realize the same operation, i.e. broadcasting lines of B_1 (resp. B_2) in column trees. Similarly, Part 1 of $\mathcal{M}_{col}$ used to compute M_1 and M_2 (resp. M_3 and M_4) broadcast columns of A_2 (resp. A_4) in line trees. Then, by computing M_1, M_4, M_2 and M_3 in this order, each of these broadcastings has to be realised only one time, which takes a global time $2T_1$ (i.e. during the computing of R_1 and R_4).

Hence, with this algorithm (which we call $Mult_2N$), we can realize in $MT(d, n)$

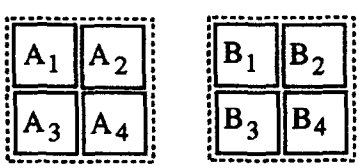


Fig. 3. Submatrices of A and B.

the product of two square matrices of order $2N = 2d^n$ in a time $\mathcal{T} = 4(T_G + t_{op}) - 2T_1 = 2T_1 + 4(T_2 + T_3 + ((d^n/n) + 1)t_{op}) \leqslant V_1 t_{op} + V_2 \beta + V_3 \tau$, with

$$V_1 = 4\left(2dN + \frac{2N}{\log_d N} + 1\right), \quad V_2 = 18 \log_d N - 8, \quad V_3 = \frac{12d - 8}{d - 1} N.$$

We finally make some remarks on the location of data of A, B and M in $MT(d, n)$, and on which way the communications are realized in this algorithm.

Remark 1. In the algorithm *Mult_2N*, each vertex $(u; v)$ of $MT(d, n)$, with $[u]_d = i$ and $[v]_d = j$ has to contain

- $A[i, j]$, $A[i + d^n, j]$, $A[j, i + d^n]$ and $A[j + d^n, i + d^n]$,
- $B[j, i]$, $B[j + d^n, i]$, $B[i + d^n, j]$ and $B[i + d^n, j + d^n]$.

For all $(m, m') \in \{0, d^n\}^2$, $(u; v)$ will finally contain the four elements $M[i + m, j + m']$.

The second remark is directly deduced from the description of the algorithm in Section 4.1.

Remark 2. In *Mult_2N*, all the communications are scheduled in steps of different time.

5. Application into the de Bruijn graph

In all the following, we consider that $\lfloor (d - 1)/2 \rfloor = k^2$, where k is an integer.

5.1. Previous results about embedding and mapping

Assume $d > 2$. We first present a $\lfloor (d - 1)/2 \rfloor$-embedding (p, R) of $MT(d, n)$ into $UB(d, 2n + 2)$ defined in [1]. Let G_M be the graph made of $\lfloor (d - 1)/2 \rfloor$ disjoint copies of $MT(d, n)$, each one denoted by $MT_\alpha(d, n)$, with $\alpha \in \{0, \ldots, \lfloor (d - 1)/2 \rfloor - 1\}$; a vertex $(u; v)$ in $MT_\alpha(d, n)$ is denoted by $(u; v)_\alpha$. Then,

$$p(u; v)_\alpha = \begin{cases} 0^{n-t} b_\alpha \bar{u} v & \text{if } |u| \geqslant |v|, \text{ with } |v| = t, \\ a_\alpha \bar{u} v b_\alpha 0^{n-t'-1} & \text{if } |u| < |v|, \text{ with } |u| = t', \end{cases}$$

where $a_\alpha = 2\alpha + 1$ and $b_\alpha = 2(\alpha + 1)$. We do not specify here the definition of R; we just recall the following result whose proof uses the embedding (p, R) [1]:

Theorem 1. *If $d > 2$ then $MT(d, n)$ can be $\lfloor (d - 1)/2 \rfloor$-embedded into $UB(d, 2n + 2)$ with dilation 3 and congestion 2.*

We give a new property of (p, R) deduced from the definition of p.

Lemma 1. *Let $(u; v)$ be a vertex of $MT(d, n)$. For all pairs of different integers α and α' in $\{0, \ldots, \lfloor (d - 1)/2 \rfloor\}$, $dist(p(u; v)_\alpha, p(u; v)_{\alpha'}) = 2$.*

We now give a result about some mappings on $UB(d, 2n+2)$ we use in the following. Let q be an integer in $\{2, \ldots, d\}$. We define an *uncomplete tree* $PT(q, k)$ as a subtree of $T(q, \mathscr{H})$, with $\mathscr{H} = \lceil \log_q k \rceil$, induced by the shortest paths from e to all the leaves m such that $[m]_d < k$ (see $PT(3, 7)$ in Fig. 4).

Lemma 2. *For each* $(u; v) \in V(MT(d, n))$, *with* $|u| = |v| = n$, *and for each* $i \in \{0, \ldots, k-1\}$, *one can map with dilation* 1 *on* $UB(d, 2n+2)$ *a tree* $T_i(u; v)$ *isomorphic to* $PT(q, k)$, *such that*

(a) *each leaf is mapped on a vertex* $b_\alpha \bar{u}v$, *where* $\alpha = ki + t$ *and* $0 \leqslant t \leqslant k-1$,

(b) *the root is mapped on a vertex at distance at most* 2 *from each vertex* $b_\alpha \bar{u}v$,

(c) *a vertex* X *in* $UB(d, 2n+2)$ *is the image of vertices of at most* 2 *different such trees* T *and* T',

(d) *if two vertices* v *and* v' *of some trees are mapped on a same vertex in* $UB(d, 2n+2)$, *then* v *and* v' *do not have the same depth in their respective trees.*

Proof. For each vertex $(u; v)$ of $MT(d, n)$, with $|u| = |v| = n$, we define k disjoint trees isomorphic to $PT(q, k)$, each one denoted $T_i(u; v)$, with $i \in \{0, \ldots, k-1\}$. We first show how we map this set of trees on $UB(d, 2n+2)$ (see Fig. 4)

- The root of $T_i(u; v)$ is mapped on $b_{ki}\bar{u}v$ if $\mathscr{H}$ is even, else on $\bar{u}vb_{ki}$.
- The vertices of depth h in $T_i(u; v)$, $0 \leqslant h \leqslant \mathscr{H}$, are mapped in vertices $b_\gamma \bar{u}v$ if $\mathscr{H} - h$ is even, else $\bar{u}vb_\gamma$, where $ki \leqslant \gamma \leqslant ki + \lceil k/q^{\mathscr{H}-h} \rceil - 1$.
- The children of a vertex of depth $h < \mathscr{H}$ mapped on $b_\gamma \bar{u}v$ (resp. $\bar{u}vb_\gamma$) are mapped in the vertices $\bar{u}vb_{\gamma'}$ (resp. $b_{\gamma'}\bar{u}v$), where $\gamma q \leqslant \gamma' \leqslant (\gamma+1)q - 1$ (note that if $\gamma = ki + \lceil k/q^{\mathscr{H}-h} \rceil - 1$ then $\gamma' \leqslant ki + \lceil k/q^{\mathscr{H}-h-1} \rceil - 1$).

(a) By construction, each leaf of $T_i(u; v)$ is mapped on a vertex $b_{ki+t}\bar{u}v$, with $0 \leqslant t \leqslant k-1$.

(b) The root of $T_i(u; v)$ is mapped on the vertex $b_{ki}\bar{u}v$ or $\bar{u}vb_{ki}$, which is at distance 0, 1 or 2 from the mapping of any leaf of $T_i(u; v)$ (depending on the parity of H).

(c) Let $X = x_1 \ldots x_{2n+2}$ be a vertex in $UB(d, 2n+2)$. The only vertices which can be mapped on X are vertices in a tree $T_i(u; v)$ with $b_{ki+t} = x_1$, and vertices in a tree $T_{i'}(u'; v')$ with $b_{ki'+t'} = x_{2n+2}$, where t and t' depend on the depth of the vertices in their respective trees.

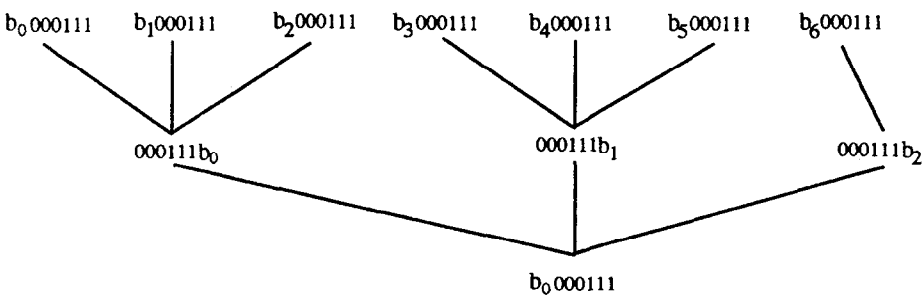


Fig. 4. Mapping of $T_0(000; 111)$ on $UB(2, 7)$.

(d) Consider that the vertex X in $UB(d, 2n+1)$ is the image of two different vertices: one vertex w of depth h in a tree $T_i(u; v)$ and another vertex w' of depth h' in a tree $T_{i'}(u'; v')$. Then, as previously in c., vertex w is mapped on $X = b_{ki+t}\bar{u}v$ with $x_1 = b_{ki+t}$ and w' is mapped on $X = \bar{u}'vb_{ki'+t'}$ with $x_{2n+1} = b_{ki'+t'}$. Hence, by definition of the mapping, h and h' do not have the same parity. □

The integer q can be considered here as a parameter. We can choose the value of q to obtain an optimal execution time for the algorithm we want to execute in $T_i(u; v)$.

5.2. An algorithm in $UB(d, 2n+2)$

Proposition 2. *The product of two square matrices A and B of order $N_T = 2kd^n$ can be realized in a time at most equal to*

$$\mathscr{V}_1 t_{op} + \mathscr{V}_2 \beta + \mathscr{V}_3 \tau, \quad \text{where} \begin{cases} \mathscr{V}_1 = 4\left(dN_T + \dfrac{2N_T}{\log_d N_T}\right) + 4k \cdot (\log_2 k + 1), \\ \mathscr{V}_2 = 108k \cdot \log_d N_T + k(\log_2 k - 46), \\ \mathscr{V}_3 = 12N_T\left(\dfrac{3d-2}{d-1}\right) + 4k(\log_2 k + 2). \end{cases}$$

in $UB(d, 2n+2)$, considering hypotheses of Section 3.1.

Hence, it is easy to see that $\mathscr{V}_1 \in \theta(dN_T)$, $\mathscr{V}_2 \in \theta(k \cdot \log_d N_T)$ and $\mathscr{V}_3 \in \theta(N_T)$.

To prove Proposition 2, we describe an algorithm realizing on $UB(d, 2n+1)$ the product of two square matrices A and B of order $2kd^n$ (remind that $k^2 = \lfloor (d-1)/2 \rfloor$). This algorithm mainly uses emulations of meshes of d-ary trees based on the embedding (p, R).

Proof. In each matrix A, B and $C = A \times B$, we consider k^2 square submatrices of order $2N$ defined as follows (we only give the definition for A). Let i and j be two integers in $\{0, \ldots, k-1\}$: each submatrix, denoted by $A_{i,j}$ is defined by

$$A_{i,j}[l, c] = A[2id^n + l, 2jd^n + c], \text{ with } (l, c) \in \{0, \ldots, k-1\}^2.$$

Hence, $C_{i,j} = \sum_{t=0}^{k-1} A_{i,t} \times B_{t,j}$.

Using the embedding (p, R) and the mapping of the trees $T_i(u; v)$, the graph $UB(d, 2n+2)$ emulates the following algorithm realizing k identical phases. The phase p, with $0 \leqslant p \leqslant k-1$, consists of:

- *Step* 1. For each $\alpha = ki + j$, $MT_\alpha(d, n)$ executes the algorithm *Mult_2N* to compute the matrix $\mathscr{M}_{ki+j} = A_{i,j} \times B_{j,i'}$, with $i' \equiv i + p \pmod{k}$.
- *Step* 2. During this step, the matrix $C_{i,j} = \sum_{t=0}^{k-1} \mathscr{M}_{ki+t}$ is computed as follows. At the end of Step 1, each vertex $(u; v)_\alpha$ in $MT_\alpha(d, n)$, contains four elements of $\mathscr{M}_\alpha$ (see Remark 1).

Therefore, using emulation in $UB(d, 2n+1)$, by Remark 1, elements $\mathcal{M}_{ki+j}[a+m, b+m']$ are contained by $P(u; v)_\alpha = b_{ki+j}\bar{u}v$, where $[u]_d = a$, $[v]_d = b$ and $(m, m') \in \{0, d^n\}^2$. This vertex is also the image of a leaf in $T_i(u; v)$. We use the mapping of this tree on $UB(d, 2n+2)$ to compute the four elements of C, each one defined by $C_{i,j}[l+a, c+b] = \sum_{t=0}^{k-1} \mathcal{M}_{ki+t}[l+a, c+b]$. These four elements are finally contained by the image of the root of $T_i(u; v)$. By Lemmas 2(a) and (b), and with the definition of the mapping we use, these elements are finally sent in two communication steps to $b_{ki+j}\bar{u}v$ in $UB(d, 2n+1)$.

In Phase 1 of the example in Fig. 5,

- $MT_3(d, n)$ computes $\mathcal{M}_3 = A_{1,0} \times B_{0,2}$,
- $MT_4(d, n)$ computes $\mathcal{M}_4 = A_{1,1} \times B_{1,2}$,
- $MT_5(d, n)$ computes $\mathcal{M}_5 = A_{1,2} \times B_{2,2}$,

and finally, $C_{1,2} = \sum_{t=0}^{2} \mathcal{M}_{3+t}$ is computed and contained by the images in $UB(d, 2n+1)$ of vertices $(u; v)$ of $MT_{3\times 1+2}(d, n)$, with $|u| = |v| = n$, by the embedding (p, R).

With the description of the algorithm and the conclusions of Section 4, we make the following remarks.

Claim 1. *In the previous algorithm,*

(a) *$MT_{ki+j}(d, n)$ has to contain matrices $A_{i,j}$ and $B_{j+p,i}$ with $p \in \{0, \ldots, k-1\}$.*

(b) *In each phase, Step 1 needs a time at most equal to $V_1 t_{op} + V_2\beta + V_3\tau$ in each graph $MT_\alpha(d, n)$.*

(c) *In each phase, Step 2 needs a time equal to $(H(\beta + 4(\tau + t_{op}))) + (2\beta + 8\tau)$ in the mapping of each tree $T_i(u; v)$ into $UB(d, 2n+2)$.*

By Remark 2 and by the description of Steps 2, we can apply here Proposition 1; so by Proposition 2, the emulation of the k Steps 1 in $UB(d, 2n+1)$ needs a time $\mathcal{S}_1 = k(V_1 t_{op} + 6(V_2\beta + V_3\tau))$. Moreover in the simulation of the Step 2 in $UB(d, 2n+1)$, by Lemmas 2(c) and (d), two vertices mapped on a same vertex in $UB(d, 2n+1)$ never do a computing or send a message simultaneously. Thus, the time required by the simulation of Step 2 in the de Bruijn graph is at most equal to $\mathcal{S}_2 = k(H(\beta + 4(\tau + t_{op})) + (2\beta + 8\tau))$.

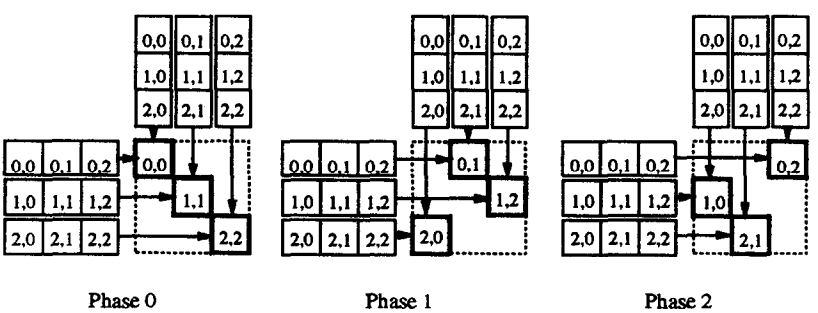


Fig. 5. An example of execution for $k = 3$.

Hence, the time required to execute the algorithm in $UB(d, 2n+1)$ is at most equal to $\mathscr{S}_1 + \mathscr{S}_2$, i.e. $k[6(V_2\beta + V_3\tau) + V_1 t_{op} + H(\beta + 4(\tau + t_{op})) + 2\beta + 8\tau]$. Therefore, the execution time of the algorithm in $UB(d, 2n+2)$ is

$$\mathscr{V}_1 t_{op} + \mathscr{V}_2 \beta + \mathscr{V}_3 \tau, \quad \text{where} \begin{cases} \mathscr{V}_1 = 8k\left(dN + \dfrac{2N}{\log_d N} + 1\right) + 4Hk, \\ \mathscr{V}_2 = 108k \cdot \log_d N - 48k + Hk + 2k, \\ \mathscr{V}_3 = 24kN\left(\dfrac{3d-2}{d-1}\right) + 4Hk + 8k. \end{cases}$$

Thus, by considering $N_T/2k = d^n$ and $\log_d N_T \simeq \log_d N$ and $H \leqslant \log_2 k$, we can say

$$\mathscr{V}_1 \leqslant 4\left(dN_T + \frac{2N_T}{\log_d N_T}\right) + 4k \cdot (\log_2 k + 1),$$

$$\mathscr{V}_2 \leqslant 108k \cdot \log_d N_T + k(\log_2 k - 46),$$

$$\mathscr{V}_3 \leqslant 12N_T\left(\frac{3d-2}{d-1}\right) + 4k(\log_2 k + 2).$$

To respect hypothesis 4 of Section 3.1, we consider that initially, elements of $A_{i,j}$ and $B_{j,i}$ are located in leaves of $MT_{ki+j}(d, n)$ as specified in Section 3.2. To obtain what is required in Claim 1(a), elements of $A_{i,j}$ have to be broadcasted to $MT_{ki+j'}(d, n)$, with $0 \leqslant j' \leqslant k-1$. By Lemma 1, we can show that this broadcasting can be done in a negligeable time compared to the time of the above algorithm. □

6. Extension to the processor bound model

The *processor bound* model of communication says that at each step, each vertex can communicate with at most one of its neighboor. This model is frequently used; we give a result about a matrices product algorithm under this model in $UB(d, 2n+1)$. We use the following lemma; we denote by $\gamma(G)$ the edge-chromatic index of G, i.e. the minimum number of colours needed to color the edges of G such that two incident edges have different colours (see [3] for precise definition).

Lemma 3. *If an algorithm needs k communication steps to run in a graph G under a link bound model, then this algorithm can be realised in $\gamma(G).k$ steps in G under a processor bound model.*

Proof. Each communication step of the initial algorithm is divided in $\gamma(G)$ steps; on each one, only communications using edges of the same color are realized. □

Corollary 1. *The products of two square matrices A and B of order* $N_T = 2kd^n$ *can be realized in a time* $\mathscr{V}_1' t_{op} + \mathscr{V}_2' \beta + \mathscr{V}_3' \tau$, *where* $\mathscr{V}_1'$ *and* $\mathscr{V}_3'$ *are in* $\theta(dN_T)$ *and* $\mathscr{V}_1'$ *is in* $\theta(d \cdot k \cdot \log_d N_T)$

Proof. It is first known that in a graph G, $\gamma(G) \leqslant \Delta(G) + 1$, where $\Delta(G)$ is the maximal degree of a vertex in G. Hence, $\gamma(UB(d, 2n+1)) \leqslant 2d + 1$. Moreover, in the above algorithm under a link bound model, communications are scheduled in steps of different times. Proposition 3 and Lemma 1 conclude. □

7. Concluding remarks

This paper leads us to make some remarks:

The constant values appearing in $\mathscr{V}_1$, $\mathscr{V}_2$ and $\mathscr{V}_3$ in Proposition 3 can be improved by considering more precisely the definition of the embedding (p, R) [1].

The algorithm we used can be efficiently extended to block-matrix product, by using pipeline communications for exchanging blocs.

The $\lceil (d-1)/2 \rceil$-embedding of $MT(d, n)$ we use gives an initial location of matrices elements in $UB(d, 2n+1)$. Moreover, Lemma 2 leads us to say that the network we emulate in $UB(d, 2n+2)$ is a collection of meshes of d-ary trees $MT(d, n)$ where the leaves of this graphs are connected by complete graphs (Fig. 6). This structure can so be used to resolve other matrices problems; the only condition is that the results of computation realized in meshes of d-ary trees must be contained by the leaves.

We can also extend this idea as follows. We try to embed in a graph H a set of copies of a graph G such that the embeddings of these copies can be *efficiently linked* in H.

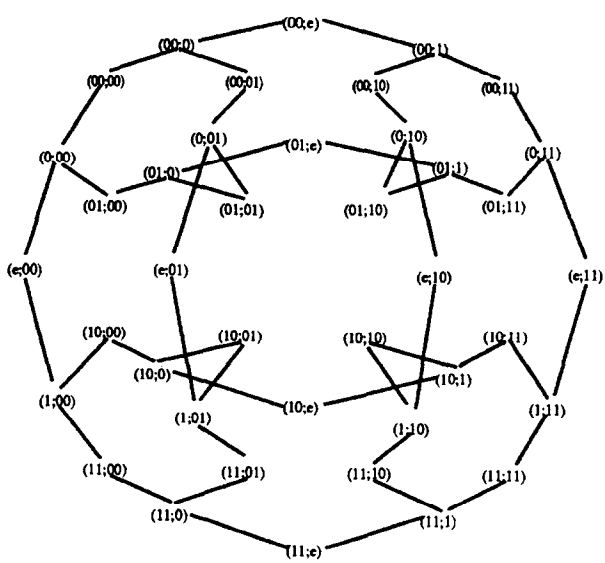


Fig. 6. $MT(2, 2)$.

References

[1] D. Barth, Embedding meshes of d-ary trees into de Bruijn graphs, *Parallel Process. Lett.* 3 (2) (1994) 115–127.

[2] D. Barth, Réseaux d'interconnexion: Structures et communications, Thesis, Université Bordeaux I, 1994.

[3] C. Berge, *Graphes et Hypergraphes* (Gauthier-Villard, Paris, 1971).

[4] J.C. Bermond and P. Fraigniaud, Communications in interconnection networks, in: *Proc. of Combinatorial Optimization in Science and Technics*, 1991.

[5] J.C. Bermond and C. Peyrat, De Bruijn and Kautz network: A competitor for the Hypercube?, in: F. André and J.P. Verjus, eds., *Proc. 1st European Workshop on Hypercube and distributed computers* (1989) 279–293.

[6] J. Bernsten, Communication efficient matrix multiplication on hypercubes, *Parallel Computing* 12 (1989) 335–342.

[7] S. Bettayeb, Z. Miller and I.H. Sudborough, Embedding grids into hypercube, in: *Proc. Agean Workshop on Computing*, Lecture Notes in Computer Science, vol. 319 (Springer, Berlin, 1988) 201–211.

[8] C. Calvin and D. Trystram, Matrix transpose for block allocations on torus and de Bruijn networks, *J. Parallel Distributed Computing* (1996), to appear.

[9] L.E. Cannon, A cellular computer to implement the Kalman filter algorithm, Ph.D. Thesis, Montana State University, 1969.

[10] M.Y. Chan, Embedding of grids into optimal hypercubes, *SIAM J. Comput.* 20 (1991) 834–864.

[11] G. Fox, M. Johnson, G. Lyzenga, S. Otto, J. Salmon and D. Walker, *Solving Problems on Concurrent Processors* (Prentice-Hall, Englewood Cliffs, NJ, 1988).

[12] P. Fraigniaud and E. Lazard, Methods and problems of communication in usual networks, *Discrete Appl. Math. 53* (1994) 79–133.

[13] M.C. Heydemann, J. Opatrny and D. Sotteau, Embeddings of hypercubes and grids into de Bruijn graphs, *J. Parallel Distributed Computing*, Proc. 1992 Internat. Conf. on Parallel Processing (1992) 28–37.

[14] R. Koch, F.T. Leighton, B. Maggs, S. Rao and A. Rosenberg, Work-preserving emulation of fixed-connection networks, in: *Proc. STOC 21th* (1989) 227–240.

[15] F.T. Leighton, *Introduction to Parallel Algorithms and Architecture: Arrays, Trees, Hypercubes* (Morgan Kaufman, Los Altos, CA, 1992).

[16] F.T. Leighton, B. Maggs and S. Rao, Packet routing and job-shop scheduling in O(congestion + dilation) steps, *Combinatorica* 14 (2) (1994) 167–186.

[17] M. Lothaire, *Combinatorics on Words*, Encyclopedia of Mathematics and its Application, Vol. 17 (Addison Wesley, Reading, MA, 1983).

[18] B. Monien and H. Sudborough, Embedding one interconnection network in another, *Computing Suppl.* 7 (1990) 257–282.

[19] J. de Rumeur, *Communications dans les Réseaux d'Interconnexion* (Masson, Paris, 1994).

[20] M.R. Samatham and D.K. Pradhan, The de Bruijn multiprocessor network: A versatile parallel processing and sorting network for VLSI, *IEEE Trans. Comput.* 38 (4) (1989).